

Kidney Stone Detection in CT with I2IReg-ClfNet

Project report: a reproduction of Öksüz et al., *Biomedical Signal Processing and Control* 119 (2026) 109857

1. What this project does

Given one axial slice of an abdominal CT scan (non-contrast), the program decides whether the slice shows a **kidney stone** or not. It first finds the kidneys, blanks out everything else, and only then looks for stones. Stone-like bright spots in other organs (gallbladder, liver, ureter, vessels), which often fool ordinary classifiers, are therefore ignored.

The project contains the full algorithm from the paper, the data download and preparation, training and evaluation scripts, a small web app, and one-click launchers for Windows and Linux. It is a research prototype and is not intended for clinical use.

2. How the algorithm works

CT slice --> [Stage 1: I2IRegNet] --> kidney-only image --> [Stage 2: ClfNet] --> Stone + / Stone
finds the kidneys looks for stones

Training loss = 0.8 x kidney-image error (half MSE) + 0.2 x classification error (cross-entropy)

Stage 1, I2IRegNet (kidney finder). A DeepLabv3+ network with a ResNet18 backbone. Instead of a yes/no mask it produces an image of the kidney region, keeping the original pixel intensities inside the kidneys so that stones stay visible. Everything outside the kidneys becomes black.

Stage 2, ClfNet (stone detector). It sees only the kidney image. Two feature extractors run side by side: a small CNN trained from scratch (fine detail) and a frozen ImageNet SqueezeNet (general shapes). Their features are merged, refined by an attention module (CBAM), and a final layer outputs the probability of a stone.

End-to-end training. Both stages are trained together with one combined loss, so the classifier's errors also improve the kidney finder.

Explanation. Grad-CAM heatmaps show which part of the image drove each decision.

Part	Parameters
Stage 1: I2IRegNet	9.8 M
Stage 2: ClfNet (shallow CNN + frozen SqueezeNet + CBAM)	2.2 M
Total	12.0 M (paper: about 13.1 M)

3. Data

Dataset	Used for	Details
KidneyStoneTR (the paper's own dataset, Figshare, CC BY 4.0)	Training and testing the stone detector	Non-contrast CT slices: 2,166 with stones, 2,018 without. Official patient-level splits: 5 cross-validation folds plus an independent hold-out test set of 732 images.
TotalSegmentator small subset (Zenodo, CC BY 4.0)	Teaching the kidney finder	CT volumes with expert kidney outlines. Only the CT and kidney files are downloaded (20-40 patients).
CT KIDNEY DATASET (Kaggle, optional)	Extra external test	Needs a Kaggle account.

Main difference from the paper. The public KidneyStoneTR release does not contain the radiologists' kidney masks, which Stage 1 needs as its target. We solved this as follows:

- Stage 1 is first trained on TotalSegmentator kidney outlines, converted to 2D slices that look like KidneyStoneTR (same orientation and CT window). Each kidney outline is filled in so that the renal pelvis, where stones usually sit, stays inside the region.
- The trained Stage 1 then draws kidney masks for KidneyStoneTR ("pseudo-masks"). 3,507 of 5,039 images received a mask; images without one simply skip the kidney part of the loss.
- The full two-stage model is then trained end-to-end on KidneyStoneTR, as in the paper.

Data issue found. 17 hold-out test images also appear in the official training folds. They are removed from training automatically to avoid leaking test data.

4. Results so far (basic model)

To confirm that the algorithm works, a **basic model** was trained on a laptop CPU: Stage 1 for 12 epochs on 20 TotalSegmentator patients, then the full model for 1 epoch on 1,000 KidneyStoneTR images. This is far less training than the paper (50 epochs on the full data with a GPU).

Check	Result
Stage 1 kidney overlap (IoU) on unseen TotalSegmentator patients	0.70 - 0.82 (Dice 0.82 - 0.90)
Model builds and learns (loss on test batch)	0.144 -> 0.014, PASS
Kidney region size on KidneyStoneTR	about 2% of the slice (two kidneys), PASS
Accuracy on 100 hold-out images (check.py)	83.0%, PASS

Full hold-out test set (732 images never seen in training):

Accuracy	Recall (sensitivity)	Specificity	Precision	F1	Cohen kappa	ROC-AUC
80.7%	76.6%	87.0%	89.9%	82.7%	0.61	0.897

	Predicted no stone	Predicted stone
True no stone (292)	254	38
True stone (440)	103	337

For comparison, the paper reports 93.3% hold-out accuracy for its fully trained model. The gap is expected after one CPU epoch; a full training run on a GPU should narrow it.

Figure 1. Stage 1 output on hold-out slices (left: input, right: predicted kidney image).

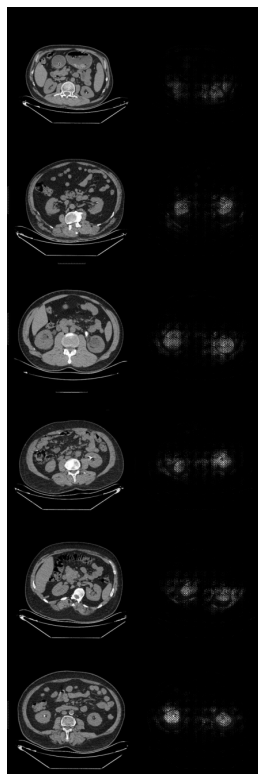
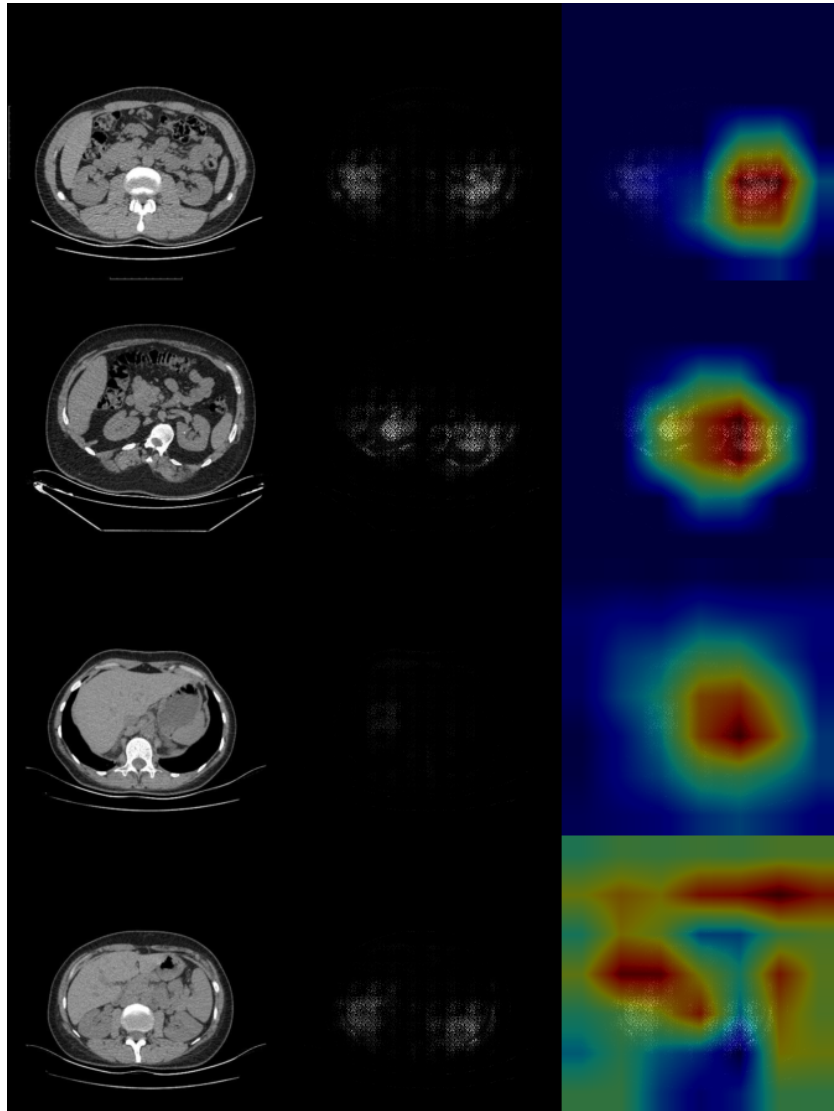


Figure 2. Example predictions: input slice, kidney image found by Stage 1, and Grad-CAM heatmap. Rows 1-2 contain stones ($P(\text{stone})$ 0.53, 0.55), rows 3-4 are normal (0.20, 0.37).



5. How to run it

	First time	Then
Windows	Install Python 3.10+ and 7-Zip, then double-click setup.bat	Double-click start.bat
Linux / macOS	Install unzip, then run ./setup.sh	Run ./start.sh

The start script shows a menu:

- [1] Check the algorithm works (about 2 min)
- [2] Open the web UI (<http://127.0.0.1:7860>)
- [3] Download datasets
- [4] Train a basic model (~2-3 h on CPU, ~15 min on a GPU)
- [5] Full paper pipeline (GPU recommended)
- [6] Accuracy on a folder of images

Accuracy on your own images: put them in sub-folders named *Positive* (stone) and *Negative* (no stone), then run `python predict_folder.py <folder>` or use the *Evaluate folder* tab of the app. PNG, JPG and DICOM are accepted.

Web app tabs

- **Detect:** click a sample or upload a slice; shows Stone +/-, the probability, the kidney image and a heatmap.
- **Evaluate folder:** accuracy, all metrics, confusion matrix and a CSV of predictions for a folder.
- **ROI-aware vs full-image:** the same slice through two models.
- **Results:** scores and training curves of finished runs.
- **Pseudo-mask review:** check and reject the automatic kidney masks.

Main files

File	Purpose
start.bat / start.sh, setup.bat / setup.sh	One-click setup and menu
app.py	Web app
check.py	Quick PASS/FAIL test of the algorithm
predict_folder.py	Accuracy on a folder of images
src/models.py	The network (I2IRegNet, ClfNet, CBAM)
scripts/	Download, data preparation, training, evaluation
samples/	8 example slices (4 stone, 4 normal)

6. Limitations and next steps

- Kidney masks are automatic (pseudo-masks), not drawn by radiologists, so kidney-overlap scores on KidneyStoneTR are only approximate.
- The current model had very little training (1 CPU epoch). Next step: the full run (start menu option 5) on a GPU.
- Stage 1 missed the kidneys on about 30% of KidneyStoneTR slices. More TotalSegmentator patients (default download: 40) and longer training should reduce this.
- Works on single 2D slices; there is no 3D or whole-scan reasoning.
- Research prototype only; not a medical device.

7. References

- C. Öksüz, A. Narter, B. Ece, M. Koyun, I. Taskent, M. K. Güllü. I2IReg-ClfNet: a cascaded multi-task deep learning framework for ROI-aware kidney stone detection in abdominal CT images. Biomedical Signal Processing and Control 119 (2026) 109857. doi:10.1016/j.bspc.2026.109857
- C. Öksüz. KidneyStoneTR dataset (2026). doi:10.6084/m9.figshare.31156783
- J. Wasserthal et al. TotalSegmentator. Radiology: Artificial Intelligence (2023). doi:10.1148/ryai.230024
- M. N. Islam et al. CT KIDNEY DATASET. Scientific Reports 12 (2022).